String Matching (2)

Peerapon S.

CPE 231 Algorithms

Topics

Boyer-Moore (BM) algorithm

String matching with finite automata

Rabin-Karp fingerprinting

Review of Horspool's Algorithm

Construct the shift table based from a pattern.

Make a largest shift possible based on the character of the text T aligning with the last character of the pattern P.

```
P = 'BARBER'
```

character c	A	B	C	D	E	F	. . .	R	. . .	Z	_
shift $t(c)$	4	2	6	6	1	6	6	3	6	6	6

```
J I M _ S A W _ M E _ I N _ A _ B A R B E R S H O P
B A R B E R             B A R B E R
        B A R B E R         B A R B E R
          B A R B E R             B A R B E R
```

Boyer-Moore (BM) Algorithm - Basic Idea

Same as Horspool's if the rightmost character in P causes the mismatch.



For $0 < k < m$ matches before a mismatch, the shift size is determined by the character in the text causing the mismatch -- ***bad-symbol shift*** and ***good-suffix shift***.

Bad-Symbol Shift

Pattern P = 'B A R B E R'

Character c	A	B	E	R	Others
Shift t(c)	4	2	1	3	6

Horspool's



Suppose $k > 0$ matches before a mismatch.

Try # shifts based on the character c in T causing the mismatch ***(the bad symbol)***

- Can we shift by $t(c)$?
- What would be the right amount of shift d_1?

Boyer-Moore



P = 'C O R N E R'

Character c	C	E	N	O	R	Others
Shift t(c)	5	1	2	4	3	6

Horspool's



Boyer-Moore



What happens if # matches $\geq t(c)$?

P = 'BARBER'

Character c	A	B	E	R	Others
Shift t(c)	4	2	1	3	6

Horspool's

```
R A R B E R _ I S _ N O T _ A _ B A R B E R
          ✗
  B A R B E R
```

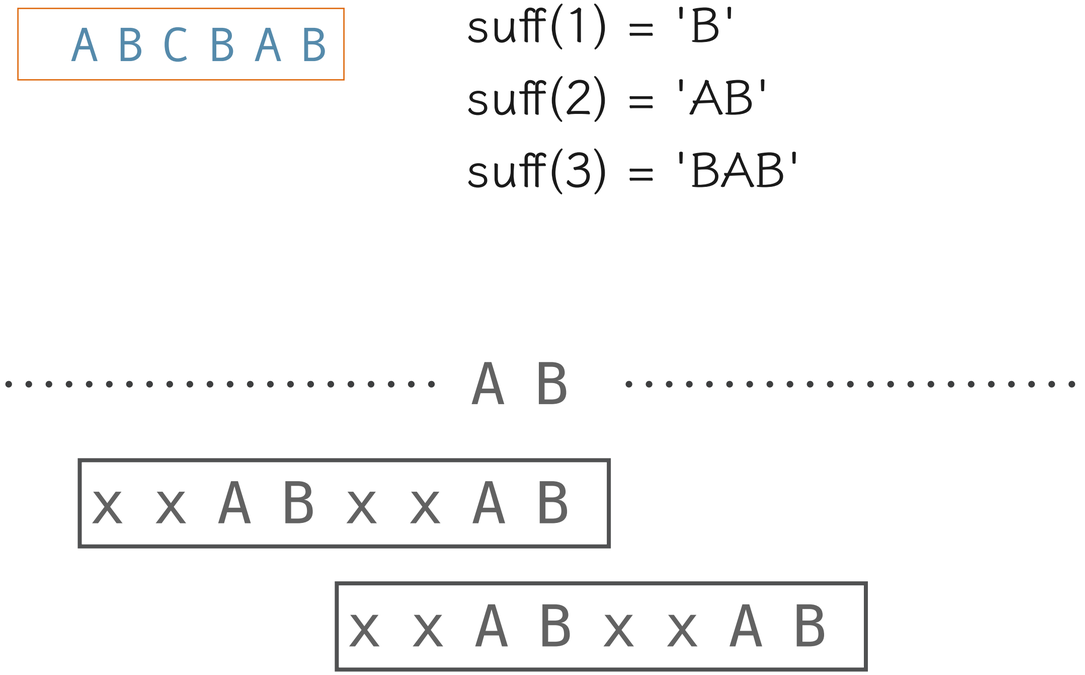
Boyer-Moore

```
R A R B E R _ I S _ N O T _ A _ B A R B E R
          ✗
  B A R B E R
```

Good-Suffix Shift

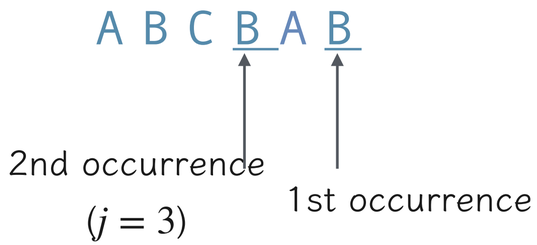
With $0 < k < m$ matches before a mismatch, denote $suff(k)$ as the last k characters of P.

Example:



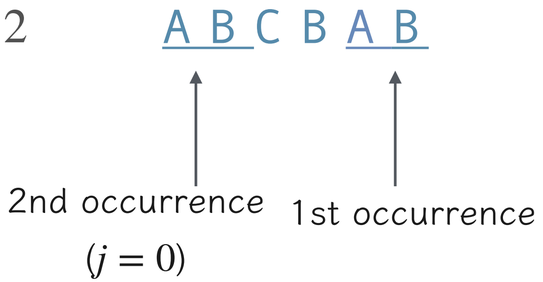
If there is another occurrence of $suff(k)$ at position j not preceded by the same character as in the rightmost occurrence, we can shift P by $d_2 = m - (j + k)$ positions between the 2nd rightmost occurrence of $suff(k)$ and its rightmost occurrence.

$k = 1$





$k = 2$

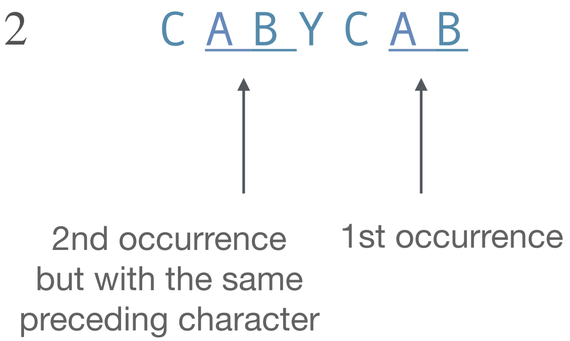


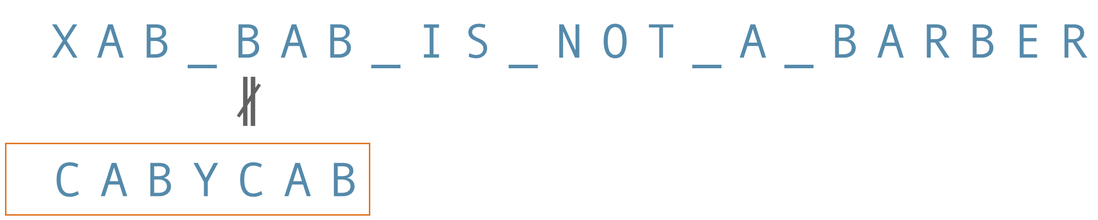


k	**pattern**	d_2
1	ABCBAB	2
2	ABCBAB	4

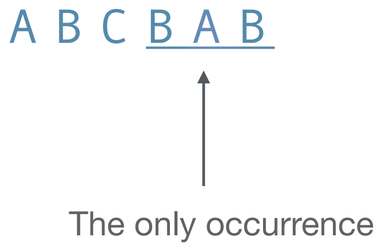
If there is no other occurrence of $suff(k)$ preceded by a different character from its rightmost occurrence, we could shift P by its entire length $m = |P|$ in most cases.

$k = 2$



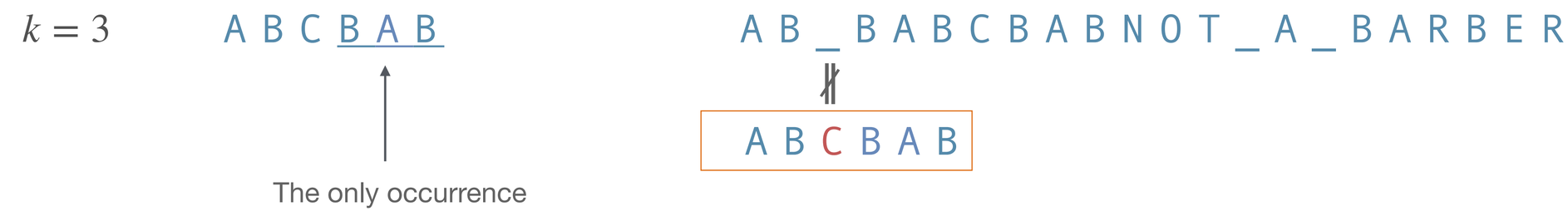


$k = 3$





However, a result from the earlier rule is not always correct.



'BAB' appears only once in P but part of it (suffix 'AB') has another occurrence in P.

If $suff(k)$ appears only once in P, determine if there exists the longest prefix of length $l < k$ in P that matches the suffix of the same size l in $suff(k)$.

Ex:

A B C B A B

$k = 3$, suff(k) = 'BAB'

So, if *suff*(k) appears only once in P and there exists prefix of length $l < k$ matching *suff*(k),

- The shift size d_2 is computed as the distance between this prefix and the corresponding suffix;
- Otherwise, d_2 is set to $m = |P|$.

A B C B A B

$k = 3$, suff(k) = 'BAB', $l = 2$



Good Suffix Shift Table

(i) If there is another occurrence of $suff(k)$ at position j not preceded by the same character as in the rightmost occurrence, $d_2 = m - (j + k)$

(ii) If $suff(k)$ appears only once in P and the prefix of length $l < k$ matching $suff(k)$ exists, $d_2 = m - l$.

(iii) Otherwise, set $d_2 = |P| = m$

k	**pattern**	d_2
1	ABC$\overline{\text{B}}$A$\underline{\text{B}}$	
2	$\overline{\text{AB}}$CB$\underline{\text{AB}}$	
3	$\overline{\text{AB}}$C$\underline{\text{BAB}}$	
4	$\overline{\text{AB}}$$\underline{\text{CBAB}}$	
5	A$\underline{\text{BCBAB}}$	

Boyer-Moore Algorithm

Construct the shift table $t(c)$

Construct the good-suffix shift table d_2

With $0 \leq k < m$ matches before a mismatch, shift the pattern to the right by the number of positions d computed by

$$d_1 = \max\{t(c) - k,\ 1\} \quad \text{(Bad-symbol shift size)}$$

$$d = \begin{cases} d_1 & k = 0 \\ \max\{d_1, d_2\} & k > 0 \end{cases}$$

Example

Consider the pattern $P =$ 'BAOBAB'

The shift table $t(c)$ for bad-symbol shifts is given by

Character c	A	B	O	Others
Shift t(c)				

Good-suffix table

k	**pattern**	d_2
1	BAO$\overline{\text{B}}$A$\underline{\text{B}}$	
2	$\overline{\text{B}}$AOB$\underline{\text{AB}}$	
3	$\overline{\text{B}}$AO$\underline{\text{BAB}}$	
4	$\overline{\text{B}}$A$\underline{\text{OBAB}}$	
5	$\overline{\text{B}}$$\underline{\text{AOBAB}}$	

B E S S _ K N E W _ A B O U T _ B A O B A B S

B A O B A B

Efficiency of Boyer-Moore Algorithm

Similar to Horspool's algorithm

- Worst-case $\Theta(nm)$ and average case $\Theta(n)$
- But space $\Theta(m + |\Sigma|)$ (instead of $\Theta(|\Sigma|)$) to store good and bad symbol shift tables.

Generally run faster than Horspool's due to the use of good and bad symbol shifts.

Perform especially well when the pattern is long and the alphabet set is large.

In-Class Assignment L9-1 (Group of 3 or less): Boyer-Moore Substring search

Suppose you want to search for a DNA gene pattern 'TGACGA' in a DNA string by using Boyer-Moore algorithm. Each position in a DNA string is taken from one of the four letters 'A', 'C', 'G', 'T'.

- Determine the shift table for bad-symbol shifts
- Determine the good-suffix shift table.
- For the DNA string TAGTAGCGATGACGACA, how many pattern shifts and comparisons does it take to find 1st appearance of the pattern? Show how the pattern was shifted during the algorithm execution.

k	Pattern	d_2
1	T G A C G A	
2	T G A C G A	
3	T G A C G A	
4	T G A C G A	
5	T G A C G A	

T A G T A G C G A T G A C G A C A

T G A C G A

Finite Automata for String Matching

```
T      A C C C A A C C A C A G T A A A A C G
       A A C C
         A A C C
             A A A C
               A A A C
```

Character c	A	C	Others
Shift t(c)	2	1	4

- Typical string matching algorithms require the whole text to be stored.
- May repeatedly backtrack the text to compare characters.

Finite automaton (FA) is a set of states and a way to go from state to state based on a sequence of inputs.

In string matching with pattern of length m,

- FA has $m + 1$ states, representing # matches.
- Input sequence = Characters from text T.

Example

- Consider $\Sigma = \{\mathtt{A}, \mathtt{C}, \mathtt{G}, \mathtt{T}\}$ and the pattern $P = \mathtt{A}\ \mathtt{A}\ \mathtt{C}$.

Next-state table

State	A	C	G	T
0				
1				
2				
3				

$next-state[s, a]$



☐ For the text $T =$ G T A A C A G T A A A C G, what are the states that FA moves ?



Algorithm FA-String-Matching

1: Input: Pattern $P = p_0 p_1 p_2 \cdots p_{m-1}$ and Text $T = t_0 t_1 t_2 \cdots t_{n-1}$, $n \geq m$
2: Output: All the shift amounts for which P occurs in T
3:
4: $Shifts \leftarrow \phi$
5: Construct the `nextstate` table from P
6: $state \leftarrow 0$
7: **for** $i := 0$ to $n - 1$ **do**
8: $\quad state \leftarrow$ `nextstate`$[state, t_i]$
9: $\quad$ **if** $state == m$ **then**
10: $\quad\quad Shifts \leftarrow Shifts \cup \{i - m + 1\}$
11: **return** $Shifts$

What is the efficiency of FA-String-Matching given the next state table ?

Determining Next-State

- Define P_k as a substring of P consisting of the first k characters, with P_0 as an empty string.

$P =$ A C A C A G A

$P_3 =$

$P_5 =$

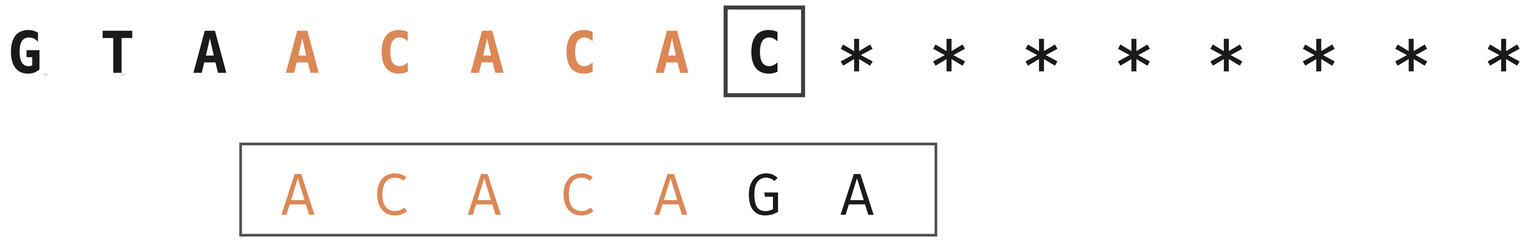
- Define the suffix of P as a substring of characters from the end of P.

- Define the ***concatenation*** of string X and a character a as a new string Xa formed by appending a to X.

Consider $\Sigma = \{\mathtt{A}, \mathtt{C}, \mathtt{G}, \mathtt{T}\}$, $P = \mathtt{A\ C\ A\ C\ A\ G\ A}$ with $k = 5$ and the next character $a = C$

Find the longest prefix of P that is a suffix of $P_k\, a$

State	A	C	G	T
0				
1				
2				
3				
4				
5				
6				
7				



Algorithm NextStateTable

Input: Pattern $P = p_0 p_1 p_2 \cdots p_{m-1}$ and Alphabet set Σ with $|\Sigma| = q$
Output: Next-state table `nextstate`

for each character $a \in \Sigma$ **do**
    for $k = 0$ to m **do**
        $j \leftarrow \min\{k+1, m\}$
        while P_j is not a suffix of $P_k a$ and $j \geq 1$ **do**
            $j \leftarrow j - 1$
        `nextstate`$[k, a] \leftarrow j$
return `nextstate`

State	A	C	G	T
0				
1				
2				
3				
4				
5				
6				
7				

What is the efficiency of filling the next state table ?

Efficiency of Finite-Automata (KMP) Algorithm

Given the next state table, FA-String-Matching is $\Theta(n)$.

Constructing the next state table takes $\Theta(qm^3)$ but can be reduced to $\Theta(m)$ in Knuth-Morris-Pratt (KMP) algorithm.

- Each character in the text is examined at most once during the search phase (never backtracks in the text).
- Appropriate for streaming texts or when texts cannot be re-examined.

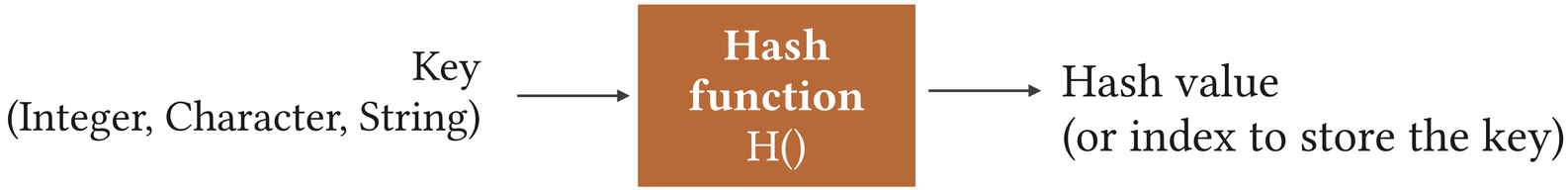
Self Test: FA for String Matching

https://forms.office.com/r/Kjve9392kE?origin=lprLink



Review of Hashing

Map a key to a big integer by using a hash function.



Simple hash functions: For a prime q and key K,

$$H(K) = K \bmod q \quad K \text{ as integer}$$

Expect to have collisions.

- Why?
- How to reduce the chance of collisions?

Example applications - Finding duplicates in a list or many lists.

String Matching using Hashing

Apply hashing to the pattern and substrings in the text, and compare them.

Text T `N O B O D Y _ N O T I C E D _ H I M`

Pattern P `N O T`

Numeric Representation of Character String

Each character in the text and pattern needs to be represented by a ***radix-d numeric value*** or ***d-ary alphabet***.

Suppose our input characters are drawn from 'A' to 'P' (16 alphabets).

Then, $\Sigma = \{0,1,2,3,\cdots,15\}$, $d = |\Sigma| = 16$

Pattern ($m = 5$): 'CABIN' => $P = p_0\, p_1\, p_2\, p_3\, p_4 \quad (p_i = P[i])$

2	0	1	8	13

Text ($n = 20$): 'PEOPLEPLANCABINPLACE' => $T = t_0\, t_1\, t_2\, t_3 \cdots t_{19} \quad (t_i = T[i])$

15	4	14	..	..	2	4

Hashing the Pattern

Map P to a decimal value by using the hash function $H(\cdot)$:

$$H(P) = p_0 d^{m-1} + p_1 d^{m-2} + \cdots + p_{m-2} d + p_{m-1} d^0$$
$$= \sum_{i=0}^{m-1} p_i \, d^{m-i-1}$$

For the pattern 'CABIN', what is the hash value p ?

2	0	1	8	13

The polynomial can be efficiently computed by using ***Horner's rule*** in $\Theta(m)$.

Horner's Rule

Problem: Given a given point x, compute the value of a nth-degree polynomial

$$p(x) = a_n x^n + a_{n-1} x^{n-1} + \cdots + a_1 x_1 + a_0$$

$$P[n] = a_n, P[n-1] = a_{n-1}, \cdots, \ P[0] = a_0$$

By successively taking x as a common factor in the remaining polynomials of diminishing degrees, it follows that

$$p(x) = (\cdots (a_n x + a_{n-1})x + \cdots)x + a_0$$

From the above form, the polynomial can be evaluated with at most n multiplications.

Let's evaluate $p(x) = 2x^4 - x^3 + 3x^2 + x - 5$ at $x = 3$:

Cofficient	P[4]=2	P[3]=-1	P[2]=3	P[1]=1	P[0]=-5
x=3	2				

ALGORITHM *Horner*$(P[0..n], x)$

//Evaluates a polynomial at a given point by Horner's rule
//Input: An array $P[0..n]$ of coefficients of a polynomial of degree n,
// stored from the lowest to the highest and a number x
//Output: The value of the polynomial at x
$p \leftarrow P[n]$
for $i \leftarrow n - 1$ **downto** 0 **do**
  $p \leftarrow x * p + P[i]$
return p

Hashing the Text

Let T_s , $s = 0,1,2,\cdots, n-m$ be a length-m substring of T starting at t_s.

Apply the same hash function to obtain the hash value $H(T_s)$ of T_s.

$P = T_s$ implies $H(P) = H(T_s)$, but not vice versa.

`Text: 'PEOPLEPLANCABINPLACE'` => $T = t_0 t_1 t_2 \cdots t_{19}$

t_0	t_1	t_2	t_3	t_4	t_5	t_6	t_7	$\cdots$						
15	4	14	15	11	4	15	11	1	13	. .	. .	. .	2	4

15	4	14	15	11	4	15	11	1	13	. .	. .	. .	2	4

15	4	14	15	11	4	15	11	1	13	. .	. .	. .	2	4

Rolling Property of Hash Function

$H(T_{s+1})$ can be computed from $H(T_s)$ in constant time thanks to the rolling property of $H(\cdot)$.

$$
\begin{aligned}
H(T_1) &= d(H(T_0) - 15d^4) + t_5 \\
H(T_2) &= d(H(T_1) - 4d^4) + t_6 \\
&\vdots \\
H(T_{s+1}) &= d(H(T_s) - t_s d^{m-1}) + t_{s+m}
\end{aligned}
$$

What is the algorithm efficiency to find all the hits ($H(P) = H(T_s)$) ?

In general, the hash values are modulo with suitable modulus q (prime) such that dq fits within a computer word.

Rabin-Karp Algorithm

Input: a string/pattern $P = p_0 p_1 \cdots p_{m-1}$ and text $T = t_0 t_1 \cdots t_{n-1}$ (radix-d numeric values)

Each character is mapped to a radix-d numeric value drawn from a range $[0, d-1]$, i.e., a finite alphabet set Σ with $d = |\Sigma|$.

For a large prime number q,

- Compute hash value of P
- Compute hash values of T_s , $s = 0,1,2,\cdots, n-m$, a length-m substring of T

For all s where $H(P) = H(T_s)$, check if the hit is **valid** or **spurious**.

Algorithm Rabin-Karp

Input: Pattern $P = p_1 p_2 \cdots p_m$, Text $T = t_1 t_2 \cdots t_n$, Radix d, Modulus q
Output: All the shifts for which P occurs in T.
$S \leftarrow \phi$
$h \leftarrow d^{m-1} \bmod q$
$H_P, H_T \leftarrow 0$
for $i = 0$ to $m - 1$ **do**
    $H_p \leftarrow (dH_P + p_i) \bmod q$
    $H_T \leftarrow (dH_T + t_i) \bmod q$
for $s = 0$ to $n - m$ **do**
    if $H_P == H_T$ and $P == T_s$ **then**
        $S \leftarrow S \cup \{s\}$
    if $s < n - m$ **then**
        $H_T \leftarrow [d(H_T - t_s h) + t_{s+m}] \bmod q$
return S

Example

Suppose characters are mapped to a radix-10 numeric value drawn from $\Sigma = \{0,1,2,\ldots,9\}$ with $d = 10$.

For the following pattern P, text T, and $q = 13$,

- What is the hash value of P ?
- Which shift sizes result in spurious hits ?

P

3	1	4	1	5

T

2	3	5	2	3	1	4	1	5	6	7	3	9	9	2	1

Self Test: Rabin-Karp String Matching

https://forms.cloud.microsoft/r/bFGVm8ifvg?origin=lprLink



Efficiency of Rabin-Karp Algorithm

Useful when searching many patterns at once.

Average-case of $\Theta(n+m)$ - Assume good hash function with few spurious hits (false positives)

- Hash value computation of P : $\Theta(m)$
- Hash value computation of T_s for $n-m+1$ times : $\Theta(n-m+1)$ by sliding through the text, computing rolling hashes and performing comparisons
- All the hits can be determined in $\Theta(m)+\Theta(n-m+1)=\Theta(n+m)$ by comparing $H(P)$ to all values of $H(T_s)$.

Worst-case of $\Theta(nm)$ - Lots of spurious hits (false positives)

- Hash value computation of P : $\Theta(m)$
- Hash value computation of T_s for $n-m+1$ times, each required $\Theta(m)$ to verify the pattern.
- All the hits can be determined in $\Theta(m)+\Theta((n-m+1)m)=\Theta(n+nm)=\Theta(nm)$
- However, with good hash function, spurious hits rarely occur.